

Distributed Monte Carlo Value-at-Risk using funcX (Globus Compute)

Roscullet Cosmin

Big Data project | Use case: Financial Analysis

1 Introducing the topic

1.1 What is the problem

Banks and investment funds hold large portfolios of shares. Every day they must answer one important question: *how much money could we lose tomorrow if the market moves against us?* This number is the portfolio's **market risk**. If it is underestimated, a single bad day can cost far more than the institution was prepared for. The most common way to measure this risk is **Value-at-Risk (VaR)**. VaR is the largest loss a portfolio is likely to suffer in one day, at a chosen confidence level. For example, a 1-day 99% VaR of \$26,000 means that on about 99 days out of 100 the loss should stay below \$26,000, and only roughly 1 day in 100 should be worse.

The difficulty is in *computing* this number. Markets are uncertain, and the shares in a portfolio move together in complicated ways. To get a trustworthy VaR we must test the portfolio against a very large number of possible “tomorrows”, and this requires a lot of computing power. That is exactly where this project starts.

1.2 Why Big Data is necessary

To measure risk we use a method called **Monte Carlo simulation**. The idea is simple: instead of trying to predict tomorrow exactly, we let the computer *invent* a very large number of possible tomorrows at random, millions of them. For each invented day we calculate how much the portfolio would gain or lose. When we put all these results together, we see the full shape of what could happen, and from that shape we read the VaR. In this project we generate **480 million** such scenarios in a single run.

This is where Big Data becomes necessary, for two reasons. First, **volume**: every risk calculation creates and processes hundreds of millions of data points. Second, **speed**: a risk team often needs the answer fast, but a single computer working alone is too slow at this scale. The good news is that each simulated “tomorrow” is completely independent from the others, so they can all be computed at the same time. This kind of problem is called *embarrassingly parallel*, and it is a perfect fit for **distributed computing**, where the work is split across many machines (nodes) that run in parallel.

1.3 What software is needed (funcX / Globus Compute)

The software used in this project is **funcX**, now called **Globus Compute**. funcX is a *Function-as-a-Service* (FaaS) platform for distributed computing. In plain words, you write a normal Python function, and funcX runs it for you on remote computers called **endpoints**, then sends the result back. You do not have to manage any servers yourself. You simply submit a function and wait for the answer, almost like calling a function on your own laptop.

funcX fits our problem very well. Because the Monte Carlo simulations are independent, we can split them into chunks and send each chunk to a different endpoint. All endpoints work at the same time, and funcX gathers the results for us. One important detail is that the submitting program (the client) and the endpoints never talk to each other directly. They communicate through the **Globus Compute cloud service**, which receives the tasks, hands them to the endpoints, and returns the results. This cloud service is what makes funcX a *serverless* and *federated* computing platform.

1.4 The benefits of modeling with Big Data technology

Building the risk system with Big Data technology brings several clear benefits.

Scalability. If we need more accuracy (more scenarios) or a faster answer, we simply add more nodes. The same code runs on one node or on many, without being rewritten.

Speed through parallelism. Splitting the work across several nodes that run at the same time can give the answer much faster than a single machine, as long as there is enough work for each node to do.

No server management. With funcX we only write the function that does the maths. The platform takes care of sending the work out, running it, and bringing the results back. This is the *serverless* idea: we focus on the computation, not on the infrastructure.

Flexibility and reproducibility. Endpoints can live anywhere (a laptop, a server, or a cloud machine) and the platform treats them all the same way. Because the function and its inputs are fixed, the same calculation can be repeated and will give the same result, which matters for a risk report that must be checked and trusted.

2 The methodology

2.1 What the system is meant to do

The system has two goals at the same time: it calculates the 1-day risk of a fixed share portfolio, and it shows how distributed computing speeds that calculation up. The portfolio is made of ten large US stocks (Apple, Microsoft, Google, Amazon, Nvidia, Meta, Tesla, JPMorgan, Exxon and Johnson & Johnson). Each holding is worth \$100,000, for a total portfolio value of \$1,000,000.

The system works in four steps. First, it downloads three years of daily prices for the ten stocks and learns from them how much each stock usually moves and how the stocks move together. Second, it creates hundreds of millions of random “next days” based on this behaviour. Third, it spreads these simulations across several funcX nodes, which run them in parallel, and then it collects the results. Fourth, from all the simulated outcomes it reads three risk numbers: the 99% VaR, the 95% VaR, and the 99% Expected Shortfall. As a separate goal, the system repeats the whole calculation with one, two, three and four nodes and records the time, so we can measure the speed-up.

2.2 Research questions and how we address them

This project tries to answer three questions.

Q1. Does the calculation get faster when we add more nodes? We answer this by running exactly the same job (480 million simulations) first on one node, then on two, three and four nodes, and by measuring the time each run takes. From these times we compute the **speed-up**, which is the one-node time divided by the time on N nodes.

Q2. How well does the work parallelise, and where does it stop helping? A perfect system would run N times faster on N nodes. In practice there is always some overhead. We measure the **efficiency**, which is the speed-up divided by the number of nodes. An efficiency near 100% means the nodes are used well, while a falling efficiency shows us where the limit is.

Q3. What is the actual risk of the portfolio? We answer this by reading the 99% and 95% VaR and the 99% Expected Shortfall from the full set of simulated outcomes, and by checking that the distributed result agrees with a smaller calculation run on a single machine.

2.3 How many nodes (funcX nodes)

In funcX, a node is an **endpoint**: a place where functions actually run. In this project each endpoint runs inside its own **Docker container**, so one container equals one node. We use up to **four nodes**, which means four containers running side by side on the same computer.

Each container is set up to behave like a single worker: it runs one task at a time and is limited to one CPU core. This choice matters for the experiment. Because every container uses exactly one core, adding a container really means adding one more unit of computing power. This lets us measure cleanly what happens to the running time as the number of nodes grows from one to four.

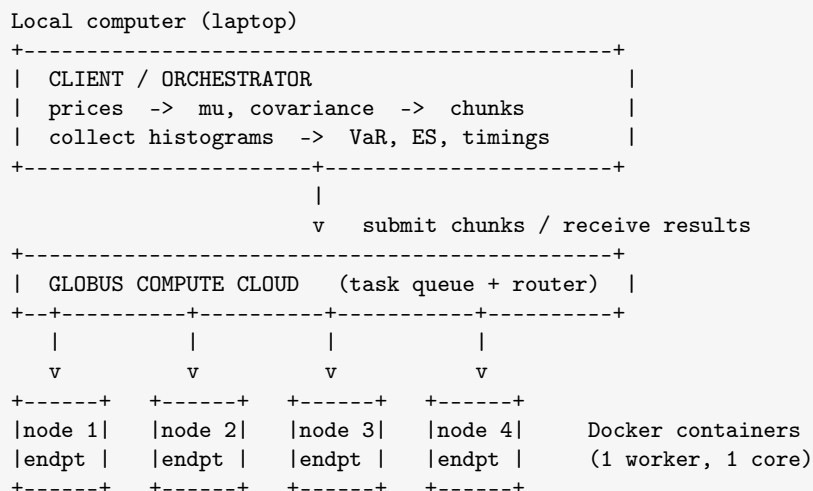
2.4 The architecture

The system has three parts: the **client**, the **cloud service**, and the **nodes**.

The **client** (also called the orchestrator) runs on the local computer. It prepares the work: it loads the prices, learns the behaviour of the stocks, splits the simulations into chunks, sends the chunks out, collects the answers, and computes the final risk numbers.

The **Globus Compute cloud service** sits in the middle. The client never contacts the nodes directly. Instead it hands every chunk to the cloud service, which keeps the chunks in a queue and passes them to whichever node is free. When a node finishes, its result travels back through the cloud service to the client.

The **nodes** are the four Docker containers. Each one runs a funcX endpoint that receives a chunk, runs the simulation, and returns a small summary of the results. The diagram below shows how the parts fit together.



2.5 Diagrams, pseudocode and workflow

The workflow follows the four steps described earlier. The pseudocode below shows the two key pieces: the function that runs on each node, and the orchestrator that drives the whole process.

The **node function** receives the description of the stocks and a number of scenarios to simulate. It draws random next-day returns that respect how the stocks move together, turns them into portfolio gains and losses, and returns a small histogram (a count of how many outcomes fell into each money bucket). Returning a histogram instead of the raw numbers keeps the message small, which is important for speed.

```
function SIMULATE_CHUNK(mu, L, positions, n_scenarios, seed):
    counts = array of zeros, one per histogram bucket
    repeat in blocks until n_scenarios are done:
        z = random normal numbers
        r = mu + z * L
        pnl = positions * (er - 1)
        add pnl into the histogram counts
    return counts
```

The **orchestrator** prepares the inputs once, then splits the total work into chunks, sends them to the nodes, adds up all the returned histograms, and reads the risk numbers from the combined histogram.

```
function RUN(nodes):
    mu, L, positions = estimate from 3 years of prices
    chunks = split TOTAL_SCENARIOS into equal pieces
    start timer
    for each chunk i:
        send SIMULATE_CHUNK to node[i mod number_of_nodes]
    total = sum of all returned histograms
    stop timer
    VaR99, VaR95, ES99 = read from total histogram
    return VaR99, VaR95, ES99, elapsed_time
```

The full experiment simply calls RUN four times, once with one node, then with two, three and four nodes, and records each time.

3 Implementation

3.1 How the implementation is deployed

The nodes are deployed with **Docker**. A small image is built from Python 3.13 and contains the funcX endpoint software and NumPy. Using **Docker Compose**, all four nodes are started with a single command, by asking Compose to scale the endpoint service to four copies. Each container starts its endpoint automatically when it boots, connects to the Globus Compute cloud, and is then ready to receive work.

A key point is how the containers log in. A funcX endpoint must prove who it is to the cloud service, and normally a person does this by clicking a login link in a browser. That does not work inside an automatic container. Instead, the project uses **client credentials**: a kind of service account made of an ID and a secret. The same ID and secret are given to every container and to the client, so they all act as the same identity and trust each other, with no human login needed. This is what allows the four nodes to start on their own and to scale up easily.

The **client** (the orchestrator) runs directly on the local computer (an Apple M4 laptop) inside a Python environment managed by the uv tool. It reads the same credentials, submits the work to the four nodes, and writes the results to disk. The actual distribution is only a few lines of real

code: for each node the client opens an executor, submits the chunks, and adds up the histograms that come back.

```
ex = globus_compute_sdk.Executor(endpoint_id=node)
futures = [ex.submit(simulate_batch, mu, L, positions, batch, i, lo, hi, n_bins)
            for i in range(n_batches)]
counts = sum(f.result() for f in futures)
```

The full source of every file is listed in the Appendix.

3.2 Which are the assumptions

The model rests on a few assumptions, and it is honest to state them clearly.

Normal returns. We assume that daily returns follow a normal (bell-shaped) distribution. This is the standard textbook assumption and it is easy to work with. In reality, markets have “fat tails”, meaning that very large crashes happen more often than a normal distribution predicts. Because of this, our VaR can understate the risk of rare, extreme days.

The past predicts the future. We learn the average return and the way the stocks move together from the last three years of prices, and we assume that tomorrow behaves in the same way. In calm periods this is reasonable, but in a crisis the behaviour can change fast, and stocks tend to fall together more than usual.

Fixed one-day positions. We assume the portfolio is not traded during the day and we look only one day ahead. The ten holdings and their values stay the same for the whole calculation.

Cached data and an available cloud. The prices are downloaded once and saved to a file, so every run uses exactly the same data and the results can be reproduced. We also assume that the internet and the Globus Compute cloud are available, because funcX always routes the work through the cloud.

3.3 System services

The running system is made of a few services that work together.

The endpoint service. Inside each container, the funcX endpoint runs as a long-living process. When it starts, it registers with the cloud and is given a unique ID. From then on it waits for tasks, runs them on its single worker, and sends the results back. Because the container keeps this process in the foreground, the node stays alive and online for as long as the container runs.

The cloud service. The Globus Compute cloud is the shared service that connects everything. It accepts tasks from the client, keeps them in a queue, gives them to free endpoints, and returns the finished results. Neither the client nor the nodes need to know each other’s network address, because the cloud handles all of this.

The orchestrator. On the local machine, the orchestrator program is the service that starts each experiment. It builds the inputs, opens a connection to each node, sends out the chunks, collects the histograms, computes the risk numbers, and saves three output files: a table of timings, the risk results, and the combined histogram used to draw the charts.

4 Validation and evaluation

4.1 Presenting the system functionality

The system was run from start to finish and worked as designed. When the orchestrator starts, it first finds the four online endpoints, then sends out the simulation chunks and waits for the

histograms to come back. After the chunks are combined, it prints the three risk numbers and saves the output files.

To make sure the maths is correct, the same calculation was also run on a single machine, without funcX, using a smaller number of scenarios. The two results agreed closely: the single-machine 99% VaR was about \$26,200, and the distributed 480-million-scenario run gave \$26,229. This match gives us confidence that splitting the work across nodes does not change the answer, it only changes the speed.

4.2 Reporting results

Risk results. Using 480 million simulated next-day outcomes, the system produced the following risk numbers for the \$1,000,000 portfolio:

Measure	Value	As % of portfolio
99% Value-at-Risk	\$26,229	2.62%
95% Value-at-Risk	\$18,316	1.83%
99% Expected Shortfall	\$30,109	3.01%

In plain words, on a normal day the portfolio is very unlikely to lose more than about \$26,000 (with 99% confidence). On the worst 1% of days, the average loss is about \$30,000. The figure below shows the full distribution of simulated outcomes, with the 99% VaR and Expected Shortfall drawn as vertical lines on the loss side.

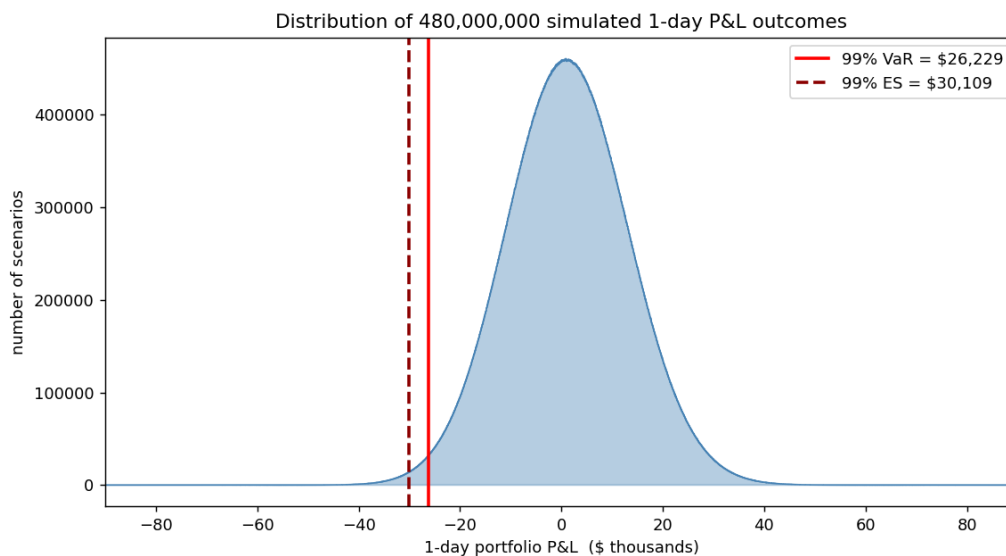


Figure 1: Distribution of 480 million simulated 1-day portfolio P&L outcomes, with the 99% VaR and the 99% Expected Shortfall marked on the loss side.

Speed results. The same 480-million-scenario job was run on one, two, three and four nodes. The table shows the time, the speed-up and the efficiency for each case.

Nodes	Time (s)	Speed-up	Efficiency
1	52.83	1.00×	100%
2	32.79	1.61×	81%
3	23.25	2.27×	76%
4	22.86	2.31×	58%

The speed-up grows steadily up to three nodes, reaching about 2.3 times faster, and then it flattens: a fourth node gives roughly the same result as three. The two charts below show the same story: the speed-up rises towards the ideal line and then levels off, while the efficiency stays above 75% up to three nodes and then falls to about 58%.

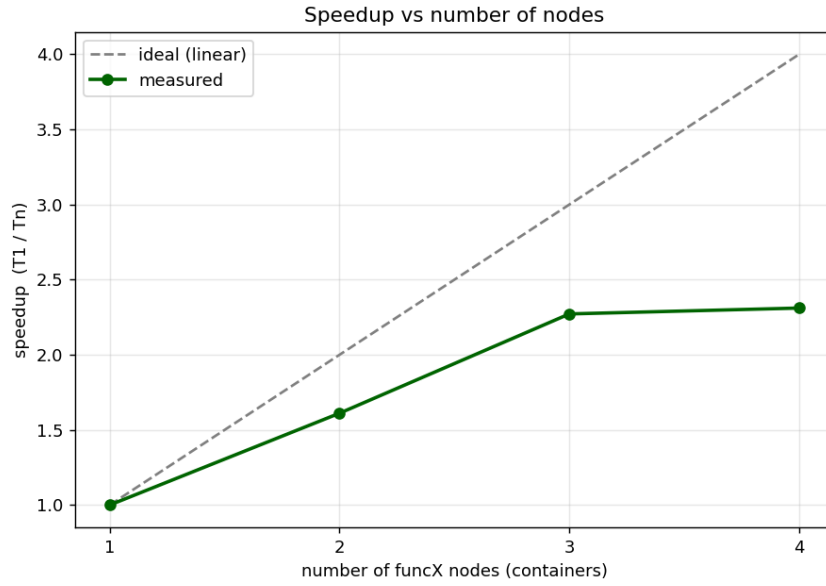


Figure 2: Speed-up versus the number of nodes, compared with the ideal (linear) line.

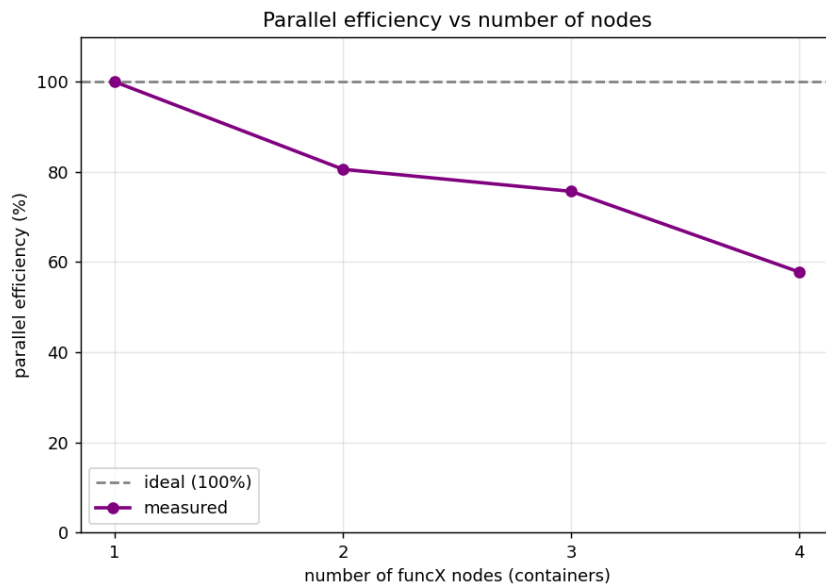


Figure 3: Parallel efficiency versus the number of nodes.

4.3 Print-screen with the system

The screenshots below were taken on the running system.

Screenshot 1. The four endpoint containers running at the same time, listed by Docker.

```
> docker compose ps
NAME                IMAGE              COMMAND              SERVICE    CREATED        STATUS        PORTS
docker-endpoint-1   docker-endpoint    "bash entrypoint.sh" endpoint    8 minutes ago  Up 8 minutes
docker-endpoint-2   docker-endpoint    "bash entrypoint.sh" endpoint    8 minutes ago  Up 8 minutes
docker-endpoint-3   docker-endpoint    "bash entrypoint.sh" endpoint    8 minutes ago  Up 8 minutes
docker-endpoint-4   docker-endpoint    "bash entrypoint.sh" endpoint    8 minutes ago  Up 8 minutes
~/Dev/M/AN1-SEM2/B/BIG_DATA_PROJECT/docker main ?9 ) _
```

Figure 4: The four endpoint containers running (`docker compose ps`).

Screenshot 2. One of the funcX endpoints inside its container, registered with the cloud and in the “Running” state.

```
> docker compose exec endpoint globus-compute-endpoint list
+-----+-----+-----+
| Endpoint ID | Status | Endpoint Name |
+-----+-----+-----+
| 7a176b2d-d254-45c6-a6d7-f9d580db80f0 | Running | mc-68ace20fd831 |
+-----+-----+-----+
~/Dev/M/AN1-SEM2/B/BIG_DATA_PROJECT/docker main ?10 ) _
```

Figure 5: A funcX endpoint registered with the cloud and running (`globus-compute-endpoint list`).

Screenshot 3. The orchestrator running the experiment: it finds the four endpoints, runs the job on one, two, three and four nodes, prints the time for each, and ends with the final risk numbers.

```
uv run python src/orchestrator.py
online endpoints: 4
283037f7-3e8d-439c-8c90-0d5d741c5c17
28c914cd-f07c-4b2e-b013-630fd020717e
7a176b2d-d254-45c6-a6d7-f9d580db80f0
983c6e3b-ee8d-415c-bba6-1c2df66ae6fe
nodes=1  sims=480,000,000  time=52.83s
nodes=2  sims=480,000,000  time=32.79s
nodes=3  sims=480,000,000  time=23.25s
nodes=4  sims=480,000,000  time=22.86s
VaR99 $26,229 (2.62%)  VaR95 $18,316  ES99 $30,109
```

Figure 6: The orchestrator running the distributed experiment and printing the timings and the risk numbers.

4.4 Explain the print-screens

Screenshot 1 proves that the deployment really is distributed. The command `docker compose ps` lists four separate containers, named endpoint-1 to endpoint-4, all in the “Up” state. Each of these is one node of the system.

Screenshot 2 shows the funcX side of a node. Running `globus-compute-endpoint list` inside a container prints the endpoint’s unique ID and its status, which is “Running”. This confirms

that the container has connected to the Globus Compute cloud and is ready to receive work. The endpoints belong to the service identity used for the client credentials, so they appear here in the terminal rather than on a personal web page.

Screenshot 3 shows the system actually working. The orchestrator first lists the four endpoint IDs it discovered, then prints one line per run with the number of nodes and the time taken, and finally prints the three risk numbers. This single screen captures both results of the project at once: the risk figures and the speed-up across nodes.

5 Conclusion

This project built and tested a small but complete Big Data system for financial risk. It computes the 1-day Value-at-Risk and Expected Shortfall of a \$1,000,000 portfolio of ten US stocks, using a Monte Carlo simulation of 480 million possible next days. The heavy work is split into chunks and run in parallel on four funcX nodes, each living in its own Docker container, while a single program on the laptop sends out the work and collects the answers.

The results answer the three research questions. The risk of the portfolio is a 99% VaR of about \$26,200 (2.62% of its value) and a 99% Expected Shortfall of about \$30,100. The distributed calculation matches a single-machine check, so we trust the numbers. On the speed side, adding nodes does make the job faster: three nodes finished the work about 2.3 times faster than one. Beyond three nodes the speed-up flattens, because all the containers share the cores of a single laptop, and once those cores are full, extra nodes cannot add more real computing power. This is a clear and honest example of how parallel speed-up is limited in practice, an effect known from the lectures as Amdahl's law and resource saturation.

The project also has limits worth naming. The risk model assumes that returns are normally distributed, which understates the chance of extreme crashes, and it assumes the recent past describes the future. The scaling test runs all nodes on one machine, so it shows the idea of distribution rather than its full power. A natural next step would be to run the funcX nodes on several separate computers, which would push the speed-up further, and to replace the normal distribution with a heavier-tailed one to capture rare events more honestly. Even so, the project shows the main lesson clearly: an embarrassingly parallel financial workload fits naturally onto a serverless, distributed platform like funcX, and Big Data tools let us compute hundreds of millions of scenarios quickly and reproducibly.

A Source code

The complete source of the project is listed below. The same code is attached to Moodle as a ZIP archive.

A.1 src/model.py : data, parameter estimation and risk from the histogram

```
1 import numpy as np
2 import pandas as pd
3
4 TICKERS = ["AAPL", "MSFT", "GOOGL", "AMZN", "NVDA", "META", "TSLA", "JPM", "XOM", "JNJ"]
5 V0 = 1_000_000.0
6 POSITIONS = np.full(len(TICKERS), V0 / len(TICKERS))
7
8
9 def load_prices(path="data/data.csv"):
10     prices = pd.read_csv(path, index_col=0, parse_dates=True)
11     return prices[TICKERS]
12
13
```

```

14 def estimate(prices):
15     returns = np.log(prices / prices.shift(1)).dropna()
16     mu = returns.mean().to_numpy()
17     cov = returns.cov().to_numpy()
18     L = np.linalg.cholesky(cov)
19     return mu, cov, L
20
21
22 def var_from_hist(counts, edges):
23     counts = np.asarray(counts, dtype=np.float64)
24     centers = (edges[:-1] + edges[1:]) / 2
25     cumulative = np.cumsum(counts)
26     total = cumulative[-1]
27     i99 = int(np.searchsorted(cumulative, 0.01 * total))
28     i95 = int(np.searchsorted(cumulative, 0.05 * total))
29     var99 = -float(centers[i99])
30     var95 = -float(centers[i95])
31     es99 = -float((centers[:i99 + 1] * counts[:i99 + 1]).sum() / cumulative[i99])
32     return var99, var95, es99

```

A.2 src/montecarlo.py : the function that runs on each funcX node

```

1 import numpy as np
2
3
4 def simulate_batch(mu, L, positions, n_sims, seed, lo, hi, n_bins):
5     import numpy as np
6     rng = np.random.default_rng(seed)
7     mu = np.asarray(mu)
8     L = np.asarray(L)
9     positions = np.asarray(positions)
10    counts = np.zeros(n_bins, dtype=np.int64)
11    done = 0
12    while done < n_sims:
13        m = min(1_000_000, n_sims - done)
14        z = rng.standard_normal((m, len(mu)))
15        returns = mu + z @ L.T
16        pnl = np.exp(m1(returns) @ positions
17        counts += np.histogram(pnl, bins=n_bins, range=(lo, hi))[0]
18        done += m
19    return counts

```

A.3 src/orchestrator.py : discover nodes, distribute, aggregate, measure

```

1 import sys
2 import os
3 import time
4 import json
5 import csv
6 import warnings
7 import numpy as np
8 import globus_compute_sdk
9 from globus_compute_sdk.serialize import ComputeSerializer, AllCodeStrategies
10 from model import load_prices, estimate, var_from_hist, POSITIONS, VO
11 from montecarlo import simulate_batch
12
13 warnings.filterwarnings("ignore", category=UserWarning)
14
15 N_SIMS = 480_000_000
16 N_BATCHES = 8
17 N_BINS = 10000
18 RESULTS = os.path.join(os.path.dirname(__file__), "..", "results")

```

```

19
20
21 def online_endpoints():
22     client = globus_compute_sdk.Client()
23     nodes = [e["uuid"] for e in client.get_endpoints()
24              if e["name"].startswith("mc-") and client.get_endpoint_status(e["uuid"])["status"]
25              == "online"]
26     return sorted(nodes)
27
28 def run(nodes, mu, L, positions, lo, hi):
29     serializer = ComputeSerializer(strategy_code=AllCodeStrategies())
30     executors = []
31     for node in nodes:
32         ex = globus_compute_sdk.Executor(endpoint_id=node)
33         ex.serializer = serializer
34         executors.append(ex)
35     batch = N_SIMS // N_BATCHES
36     start = time.perf_counter()
37     futures = [executors[i % len(executors)].submit(simulate_batch, mu, L, positions, batch, i,
38              lo, hi, N_BINS)
39               for i in range(N_BATCHES)]
40     counts = sum(f.result() for f in futures)
41     elapsed = time.perf_counter() - start
42     for ex in executors:
43         ex.shutdown()
44     return counts, elapsed
45
46 def save(timings, counts, edges, var99, var95, es99):
47     os.makedirs(RESULTS, exist_ok=True)
48     base = timings[0][1]
49     with open(os.path.join(RESULTS, "timings.csv"), "w", newline="") as f:
50         writer = csv.writer(f)
51         writer.writerow(["nodes", "time_s", "speedup", "efficiency"])
52         for n, t in timings:
53             writer.writerow([n, round(t, 3), round(base / t, 3), round(base / t / n, 3)])
54     with open(os.path.join(RESULTS, "var_results.json"), "w") as f:
55         json.dump({"portfolio_value": V0, "n_sims": N_SIMS, "VaR99": var99,
56                  "VaR95": var95, "ES99": es99, "VaR99_pct": var99 / V0 * 100}, f, indent=2)
57     np.savez(os.path.join(RESULTS, "histogram.npz"), counts=counts, edges=edges)
58
59
60 def main():
61     nodes = sys.argv[1:] or online_endpoints()
62     print("online endpoints:", len(nodes))
63     for node in nodes:
64         print(" ", node)
65
66     mu, cov, L = estimate(load_prices())
67     sigma = float(np.sqrt(POSITIONS @ cov @ POSITIONS))
68     edges = np.linspace(-12 * sigma, 12 * sigma, N_BINS + 1)
69     lo, hi = float(edges[0]), float(edges[-1])
70     mu, L, positions = mu.tolist(), L.tolist(), POSITIONS.tolist()
71
72     timings = []
73     counts = None
74     for n in range(1, len(nodes) + 1):
75         counts, elapsed = run(nodes[:n], mu, L, positions, lo, hi)
76         timings.append((n, elapsed))
77         print(f"nodes={n}   sims={N_SIMS:,}   time={elapsed:.2f}s")
78
79     var99, var95, es99 = var_from_hist(counts, edges)

```

```

80     print(f"VaR99 ${var99:,.0f} ({var99 / V0 * 100:,.2f}%)   VaR95 ${var95:,.0f}   ES99 ${es99:,.0f}")
81     save(timings, counts, edges, var99, var95, es99)
82
83
84 if __name__ == "__main__":
85     main()

```

A.4 src/plots.py : the three result charts

```

1  import os
2  import json
3  import csv
4  import numpy as np
5  import matplotlib
6  matplotlib.use("Agg")
7  import matplotlib.pyplot as plt
8
9  RESULTS = os.path.join(os.path.dirname(__file__), "..", "results")
10 FIG = os.path.join(RESULTS, "figures")
11
12
13 def load_timings():
14     with open(os.path.join(RESULTS, "timings.csv")) as f:
15         return [(int(r["nodes"]), float(r["time_s"]), float(r["speedup"]), float(r["efficiency"]
16             )))
17                 for r in csv.DictReader(f)]
18
19 def pnl_histogram():
20     d = np.load(os.path.join(RESULTS, "histogram.npz"))
21     counts, edges = d["counts"], d["edges"]
22     centers = (edges[:-1] + edges[1:]) / 2 / 1000.0
23     with open(os.path.join(RESULTS, "var_results.json")) as f:
24         res = json.load(f)
25         var99 = res["VaR99"] / 1000.0
26         es99 = res["ES99"] / 1000.0
27         plt.figure(figsize=(9, 5))
28         plt.fill_between(centers, counts, step="mid", alpha=0.4, color="steelblue")
29         plt.plot(centers, counts, drawstyle="steps-mid", color="steelblue", lw=0.8)
30         plt.axvline(-var99, color="red", lw=2, label=f"99% VaR = ${res['VaR99']:.0f}")
31         plt.axvline(-es99, color="darkred", lw=2, ls="--", label=f"99% ES = ${res['ES99']:.0f}")
32         plt.xlabel("1-day portfolio P&L ($ thousands)")
33         plt.ylabel("number of scenarios")
34         plt.title(f"Distribution of {res['n_sims']:,} simulated 1-day P&L outcomes")
35         plt.xlim(-90, 90)
36         plt.legend()
37         plt.tight_layout()
38         plt.savefig(os.path.join(FIG, "pnl_histogram.png"), dpi=130)
39         plt.close()
40
41
42 def speedup_chart():
43     rows = load_timings()
44     nodes = [r[0] for r in rows]
45     speedup = [r[2] for r in rows]
46     plt.figure(figsize=(7, 5))
47     plt.plot(nodes, nodes, "--", color="gray", label="ideal (linear)")
48     plt.plot(nodes, speedup, "o-", color="darkgreen", lw=2, label="measured")
49     plt.xlabel("number of funcX nodes (containers)")
50     plt.ylabel("speedup (T1 / Tn)")
51     plt.title("Speedup vs number of nodes")
52     plt.xticks(nodes)

```

```

53 plt.legend()
54 plt.grid(alpha=0.3)
55 plt.tight_layout()
56 plt.savefig(os.path.join(FIG, "speedup.png"), dpi=130)
57 plt.close()
58
59
60 def efficiency_chart():
61     rows = load_timings()
62     nodes = [r[0] for r in rows]
63     eff = [r[3] * 100 for r in rows]
64     plt.figure(figsize=(7, 5))
65     plt.axhline(100, ls="--", color="gray", label="ideal (100%)")
66     plt.plot(nodes, eff, "o-", color="purple", lw=2, label="measured")
67     plt.xlabel("number of funcX nodes (containers)")
68     plt.ylabel("parallel efficiency (%)")
69     plt.title("Parallel efficiency vs number of nodes")
70     plt.xticks(nodes)
71     plt.ylim(0, 110)
72     plt.legend()
73     plt.grid(alpha=0.3)
74     plt.tight_layout()
75     plt.savefig(os.path.join(FIG, "efficiency.png"), dpi=130)
76     plt.close()
77
78
79 if __name__ == "__main__":
80     os.makedirs(FIG, exist_ok=True)
81     pnl_histogram()
82     speedup_chart()
83     efficiency_chart()
84     print("saved figures to", FIG)

```

A.5 data/prices.py : download and cache the price data

```

1 import yfinance as yf
2
3 TICKERS = ["AAPL", "MSFT", "GOOGL", "AMZN", "NVDA", "META", "TSLA", "JPM", "XOM", "JNJ"]
4
5 raw_data = yf.download(TICKERS, period="3y", interval="1d", auto_adjust=True)
6 raw_data = raw_data["Close"][TICKERS]
7 raw_data = raw_data.dropna()
8 raw_data.to_csv("data.csv")
9
10 print("shape:", raw_data.shape)
11 print("range:", raw_data.index.min().date(), raw_data.index.max().date())
12 print(raw_data.tail(3))

```

A.6 docker/Dockerfile : the endpoint image

```

1 FROM python:3.13-slim
2
3 RUN apt-get update && apt-get install -y --no-install-recommends build-essential && rm -rf /var
  /lib/apt/lists/*
4 RUN pip install --no-cache-dir globus-compute-endpoint "numpy==2.4.6"
5
6 RUN useradd -m compute
7 USER compute
8 WORKDIR /home/compute
9
10 COPY --chown=compute:compute user_config_template.yaml.j2 entrypoint.sh ./
11

```

```
12 ENTRYPOINT ["bash", "entrypoint.sh"]
```

A.7 docker/docker-compose.yml : start and scale the nodes

```
1 services:
2   endpoint:
3     build: .
4     cpus: 1.0
5     env_file:
6       - .env
7     environment:
8       OMP_NUM_THREADS: "1"
9       OPENBLAS_NUM_THREADS: "1"
10      MKL_NUM_THREADS: "1"
```

A.8 docker/entrypoint.sh : configure and start the endpoint

```
1 #!/usr/bin/env bash
2 set -e
3 NAME="mc-$(hostname)"
4 globus-compute-endpoint configure "$NAME"
5 cp /home/compute/user_config_template.yaml.j2 "$HOME/.globus_compute/$NAME/user_config_template
6   .yaml.j2"
7 exec globus-compute-endpoint start "$NAME"
```

A.9 docker/user_config_template.yaml.j2 : the endpoint engine config

```
1 engine:
2   type: GlobusComputeEngine
3   max_workers_per_node: 1
4   provider:
5     type: LocalProvider
6     init_blocks: 1
7     min_blocks: 0
8     max_blocks: 1
```